



Rapport De Soutenance

Sujet

Création d'un site Web QCM

Module

Développement Web

Période :

Du 16/05/2026 Au
27/05/2026

Membres Du Groupe

Morsi Leticia

Dellaai Yassmine

2025/2026

Supervisée par

Mr GUEZ

INTRODUCTION

Dans le cadre de l'année scolaire 2025–2026, nous avons réalisé un projet de développement web ayant pour objectif la conception et la mise en place d'une application de QCM en ligne inspirée des plateformes d'évaluation numérique. Ce projet vise à reproduire les principales fonctionnalités d'une plateforme de questionnaires à choix multiples, permettant aux utilisateurs de s'inscrire, se connecter, passer des QCM par catégories, consulter leurs résultats ainsi que leur historique. L'application inclut également des fonctionnalités avancées telles qu'un système anti-triche complet (plein écran obligatoire, détection des changements d'onglet, blocage du clic droit et des raccourcis clavier), ainsi qu'une interface d'administration pour la gestion des questions, des utilisateurs et la surveillance des tentatives de triche.

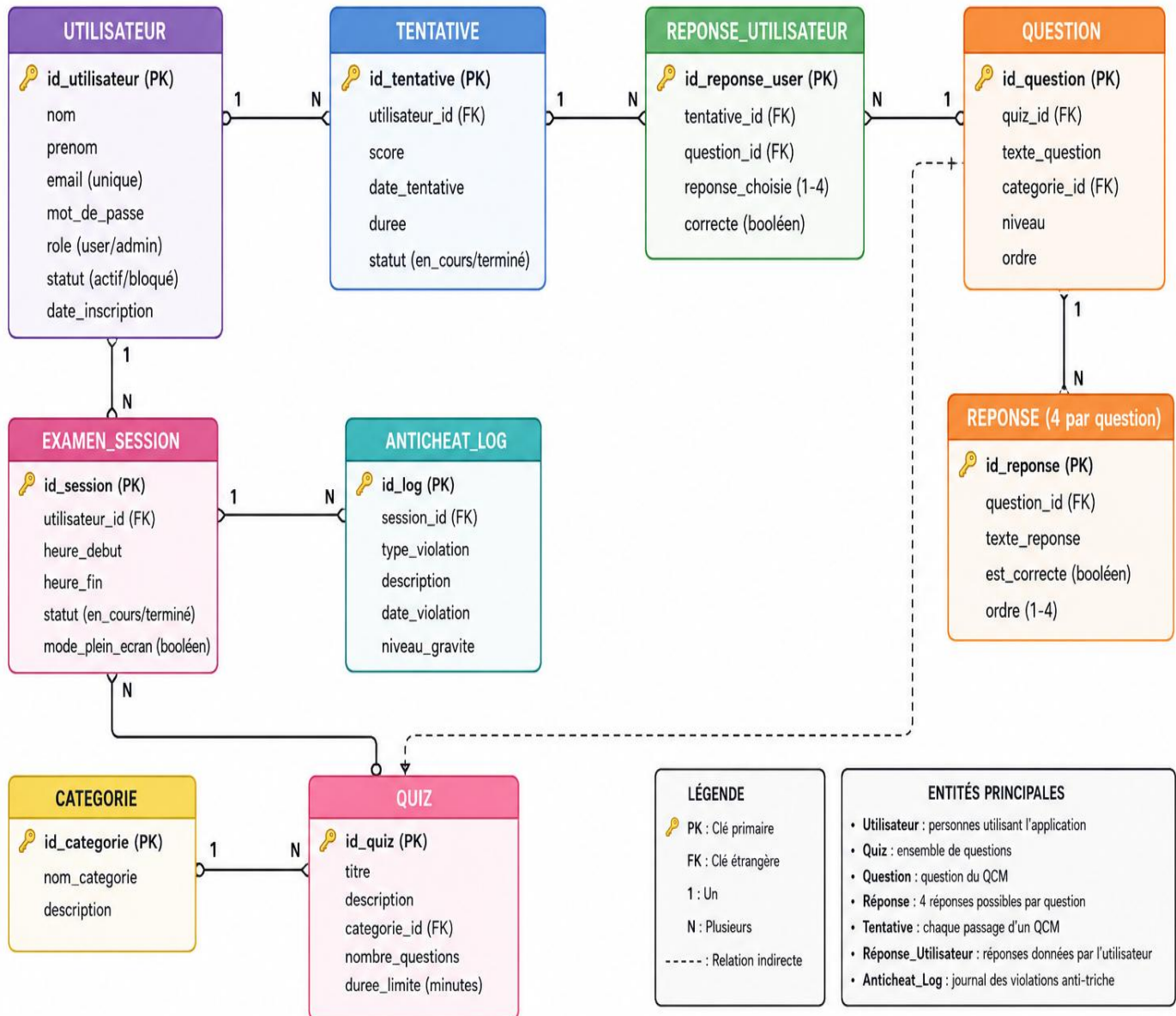
Le développement de ce projet repose sur l'utilisation de technologies web courantes, notamment PHP pour la partie serveur, HTML et CSS pour l'interface utilisateur, JavaScript pour l'amélioration de l'interactivité et la détection des comportements suspects, ainsi que SQL pour la gestion et le stockage des données (utilisateurs, questions, tentatives, logs anti-triche).

Ce projet s'inscrit dans une démarche pédagogique visant à mettre en pratique les compétences acquises en développement web, en conception de bases de données et en organisation de projet. Il permet également de simuler le fonctionnement d'une application réelle en respectant les bonnes pratiques de sécurité et de développement.

Ce rapport présente l'ensemble des étapes de réalisation du projet, incluant l'analyse des besoins, la conception de la base de données (schéma relationnel et script SQL), l'implémentation des fonctionnalités (frontend et backend), le système anti-triche, ainsi que les outils utilisés, la méthodologie de travail adoptée et les perspectives d'évolution de l'application.

II. Conception 1- Model Conceptuel des données (MCD)

MODÈLE CONCEPTUEL DE DONNÉES (MCD) - APPLICATION QCM



MODÈLE CONCEPTUEL DE DONNÉES (MCD)

Dans le cadre de la conception de notre plateforme de QCM, nous avons élaboré un Modèle Conceptuel de Données (MCD) afin de structurer et organiser les informations essentielles du système. Ce modèle permet de représenter de manière claire et visuelle les entités principales de l'application ainsi que les relations qu'elles entretiennent entre elles.

Le MCD que nous avons réalisé met en évidence huit entités principales :

1. **Utilisateur** : représente les personnes utilisant la plateforme (étudiants ou administrateurs). Chaque utilisateur possède un profil (nom, prénom, email, mot de passe) ainsi qu'un rôle (user ou admin). Il peut également être bloqué en cas de comportement suspect.
2. **Catégorie** : regroupe les quiz par thème (Informatique, Mathématiques, Culture générale, etc.). Une catégorie peut contenir plusieurs quiz.
3. **Quiz** : représente un ensemble de questions à choix multiples. Chaque quiz est rattaché à une catégorie spécifique et peut contenir plusieurs questions.
4. **Question** : constitue l'unité de base d'un quiz. Chaque question propose quatre réponses possibles (reponse1 à reponse4) et une seule bonne réponse (bonne_reponse). Une question appartient obligatoirement à un quiz.
5. **Tentative** : enregistre le passage d'un QCM par un utilisateur. Une tentative stocke le score obtenu, la date de passage et le temps total pris.
6. **Réponse_Utilisateur** : permet de conserver le détail des réponses données par l'utilisateur pour chaque question lors d'une tentative. Cette entité permet de savoir si la réponse était correcte et le temps pris pour répondre.
7. **Examen_Session** : entité dédiée au système anti-triche. Elle suit en temps réel la session active d'un utilisateur pendant qu'il passe un QCM. Elle enregistre l'heure de début, le score de suspicion cumulé, le statut de la session (active, terminée, invalidée) ainsi que le dernier signe de vie (heartbeat).
8. **AntiCheat_Log** : stocke tous les événements suspects détectés pendant une session (changement d'onglet, sortie du plein écran, tentative de copier/coller, ouverture de la console développeur, etc.). Chaque log est associé à une session et à un utilisateur.

Les relations entre ces entités sont les suivantes :

- Un utilisateur peut passer plusieurs tentatives (relation 1,n)
- Un utilisateur peut avoir plusieurs sessions d'examen (relation 1,n)
- Un utilisateur peut générer plusieurs logs anti-triche (relation 1,n)
- Une catégorie contient plusieurs quiz (relation 1,n)
- Un quiz contient plusieurs questions (relation 1,n)
- Un quiz peut être passé plusieurs fois via les tentatives (relation 1,n)
- Une tentative contient plusieurs réponses utilisateur (relation 1,n)
- Une question peut avoir plusieurs réponses utilisateur (relation 1,n)
- Une examen_session peut générer plusieurs logs anti-triche (relation 1,n)

Ce modèle conceptuel de données garantit la cohérence, l'intégrité et la bonne organisation des données au sein de l'application. Il permet également de gérer efficacement le système anti-triche, le suivi des performances

MODÈLE LOGIQUE DE DONNÉES (MLD) – PLATEFORME QCM "LETI QCM"

À partir du Modèle Conceptuel de Données (MCD), nous avons dérivé le Modèle Logique de Données (MLD) permettant de représenter la structure de la base de données sous forme relationnelle. Ce modèle a ensuite été directement implémenté dans la base de données de l'application.

STRUCTURE DES TABLES – PLATEFORME QCM "LETI QCM"

UTILISATEUR(#id, nom, prenom, email, mot_de_passe, role, est_bloque, created_at)

CATEGORIE(#id, nom, description, icone, created_at)

QUIZ(#id, titre, description, categorie_id*, duree_limite, est_actif, created_at)

QUESTION(#id, quiz_id*, texte, reponse1, reponse2, reponse3, reponse4, bonne_reponse, points, ordre)

TENTATIVE(#id, utilisateur_id, quiz_id, score, total_questions, date, temps_total)

REPONSE_UTILISATEUR(#id, tentative_id, question_id, reponse_utilisateur, est_correcte, temps_reponse)

EXAMEN_SESSION(#id, utilisateur_id*, start_time, end_time, suspicion_score, status, last_heartbeat)

ANTICHEAT_LOG(#id, session_id, utilisateur_id, event_type, event_data, created_at)

Clé primaire

* Clé étrangère

DESCRIPTION DES RELATIONS – PLATEFORME QCM "LETI QCM"

La table **TENTATIVE** est liée à la table **UTILISATEUR** via la clé étrangère `utilisateur_id`. Un utilisateur peut donc passer plusieurs tentatives de QCM, mais chaque tentative appartient à un seul utilisateur.

La table **REPONSE_UTILISATEUR** constitue une table d'association entre **TENTATIVE** et **QUESTION**, permettant de stocker le détail des réponses données par l'utilisateur pour chaque question lors d'une tentative spécifique.

La table **QUIZ** est liée à la table **CATEGORIE** via la clé étrangère `categorie_id`. Une catégorie peut contenir plusieurs quiz, mais chaque quiz appartient à une seule catégorie.

La table **QUESTION** est liée à la table **QUIZ** via la clé étrangère `quiz_id`. Un quiz peut contenir plusieurs questions, mais chaque question appartient à un seul quiz.

La table **EXAMEN_SESSION** est liée à la table **UTILISATEUR** via la clé étrangère `utilisateur_id`. Un utilisateur peut avoir plusieurs sessions d'examen (notamment en cas de reprise), mais chaque session appartient à un seul utilisateur.

La table **ANTICHEAT_LOG** est liée à la table **EXAMEN_SESSION** via la clé étrangère `session_id`, et à la table **UTILISATEUR** via la clé étrangère `utilisateur_id`. Une session peut générer plusieurs logs anti-triche, et chaque log est associé à un seul utilisateur. Cette table permet de tracer tous les événements suspects détectés pendant un QCM (changement d'onglet, sortie du plein écran, tentative de copier/coller, ouverture de la console développeur, etc.).

CONCLUSION

Ce Modèle Logique de Données (MLD) assure une organisation cohérente et normalisée des données de la plateforme QCM. Il permet de garantir l'intégrité des relations entre les différentes entités (Utilisateur, Catégorie, Quiz, Question, Tentative, Réponse_Utilisateur, Examen_Session et AntiCheat_Log) et a été directement utilisé pour la création de la base de données du projet.

Grâce à cette structure, l'application peut gérer efficacement les utilisateurs, les QCM, les tentatives, ainsi que le système anti-triche avec le suivi des sessions et des logs de comportements suspects. Ce modèle garantit également la cohérence des données, évite les redondances et facilite les évolutions futures de l'application.

SCRIPT SQL DE CRÉATION DE LA BASE DE DONNÉES

À partir du Modèle Logique de Données (MLD), un script SQL a été généré afin d'implémenter la base de données sur un serveur MySQL. Ce script permet de créer l'ensemble des tables nécessaires au fonctionnement de l'application ainsi que les relations entre elles.

Table	Rôle
utilisateur	Stocke les comptes utilisateurs avec email unique, mot de passe chiffré, nom, prénom, rôle (user/admin), statut de blocage et date d'inscription
categorie	Stocke les catégories de QCM (Informatique, Maths, Culture générale, etc.) avec nom, description et icône
quiz	Contient les QCM, liés à une catégorie via <code>categorie_id</code> , avec titre, description et durée limite
question	Contient les questions, liées à un quiz via <code>quiz_id</code> , avec énoncé, quatre réponses (<code>reponse1</code> à <code>reponse4</code>) et la bonne réponse
tentative	Enregistre les tentatives des utilisateurs, liée à un utilisateur et à un quiz, avec score, nombre de questions et date
reponse_utilisateur	Stocke le détail des réponses données par l'utilisateur, liée à une tentative et à une question
examen_session	Stocke les sessions d'examen actives pour l'anti-triche, liée à un utilisateur, avec score de suspicion et statut
anticheat_log	Contient les logs des tentatives de triche, liés à une session et à un utilisateur, avec type d'événement et date

2. CONTRAINTES D'INTÉGRITÉ

Plusieurs contraintes ont été mises en place afin de garantir la cohérence des données :

- **Clés étrangères** : Toutes les clés étrangères utilisent la contrainte ON DELETE CASCADE afin de supprimer automatiquement les données liées (ex : suppression d'un utilisateur entraîne la suppression de ses tentatives et sessions).
- **Contrainte d'unicité** : Un email ne peut être utilisé qu'une seule fois dans la table utilisateur (UNIQUE).
- **Contraintes CHECK** : Le champ bonne_reponse dans la table question est limité aux valeurs 1, 2, 3 ou 4.
- **Valeurs par défaut** : Les champs comme role (user), est_bloque (0), status (active) et suspicion_score (0) ont des valeurs par défaut.
- **Index** : Des index ont été créés sur certains champs afin d'améliorer les performances des requêtes, notamment pour la recherche, les filtres et la gestion des logs anti-triche.

3. CHOIX TECHNIQUES

Plusieurs choix techniques ont été effectués lors de la conception de la base de données :

- **Moteur InnoDB** : Utilisé afin de garantir la gestion des transactions et l'intégrité référentielle.
- **Type ENUM** : Utilisé pour les champs à valeurs limitées (role : user/admin, status : active/finished/invalidated).
- **Type DATETIME** : Utilisé pour assurer le suivi et l'historisation des dates (création, modification, connexion, tentative).
- **Hashage des mots de passe** : Utilisation de password_hash() en PHP pour stocker les mots de passe de manière sécurisée.
- **Requêtes préparées** : Utilisées pour prévenir les injections SQL.
- **Types TEXT** : Utilisés pour les champs contenant de longs textes (description des quiz, énoncé des questions, logs).

```
-- =====  
-- BASE DE DONNÉES : LETI QCM  
-- =====
```

```
-- -----  
-- 1. TABLE UTILISATEUR  
-- -----
```

```
CREATE TABLE IF NOT EXISTS utilisateur (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nom VARCHAR(100) NOT NULL,  
    prenom VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    mot_de_passe VARCHAR(255) NOT NULL,  
    role ENUM('user', 'admin') DEFAULT 'user',  
    est_bloque TINYINT(1) DEFAULT 0,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- -----  
-- 2. TABLE CATEGORIE  
-- -----
```

```
CREATE TABLE IF NOT EXISTS categorie (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nom VARCHAR(100) UNIQUE NOT NULL,  
    description TEXT,  
    icone VARCHAR(50) DEFAULT 'fa-book',  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

-- 3. TABLE QUIZ

```
CREATE TABLE IF NOT EXISTS quiz (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    titre VARCHAR(200) NOT NULL,  
    description TEXT,  
    categorie_id INT NOT NULL,  
    duree_limite INT DEFAULT 600,  
    est_actif TINYINT(1) DEFAULT 1,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (categorie_id) REFERENCES categorie(id) ON DELETE CASCADE  
);
```

-- 4. TABLE QUESTION

```
CREATE TABLE IF NOT EXISTS question (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    quiz_id INT NOT NULL,  
    texte TEXT NOT NULL,  
    reponse1 VARCHAR(500) NOT NULL,  
    reponse2 VARCHAR(500) NOT NULL,  
    reponse3 VARCHAR(500) NOT NULL,  
    reponse4 VARCHAR(500) NOT NULL,  
    bonne_reponse TINYINT(1) NOT NULL CHECK (bonne_reponse BETWEEN 1 AND 4),  
    points INT DEFAULT 1,  
    ordre INT DEFAULT 0,  
    FOREIGN KEY (quiz_id) REFERENCES quiz(id) ON DELETE CASCADE  
);
```

-- -----

-- 5. TABLE TENTATIVE

-- -----

```
CREATE TABLE IF NOT EXISTS tentative (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    utilisateur_id INT NOT NULL,  
    quiz_id INT NOT NULL,  
    score INT DEFAULT 0,  
    total_questions INT NOT NULL,  
    date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    temps_total INT,  
    FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id) ON DELETE CASCADE,  
    FOREIGN KEY (quiz_id) REFERENCES quiz(id) ON DELETE CASCADE  
);
```

-- -----

-- 6. TABLE REPONSE_UTILISATEUR

-- -----

```
CREATE TABLE IF NOT EXISTS reponse_utilisateur (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    tentative_id INT NOT NULL,  
    question_id INT NOT NULL,  
    reponse_utilisateur INT CHECK (reponse_utilisateur BETWEEN 1 AND 4),  
    est_correcte TINYINT(1) DEFAULT 0,  
    temps_reponse INT,  
    FOREIGN KEY (tentative_id) REFERENCES tentative(id) ON DELETE CASCADE,  
    FOREIGN KEY (question_id) REFERENCES question(id) ON DELETE CASCADE  
);
```

```
-- -----  
-- 7. TABLE EXAMEN_SESSION (anti-triche)  
-- -----
```

```
CREATE TABLE IF NOT EXISTS examen_session (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    utilisateur_id INT NOT NULL,  
    start_time DATETIME NOT NULL,  
    end_time DATETIME,  
    suspicion_score INT DEFAULT 0,  
    status ENUM('active', 'finished', 'invalidated') DEFAULT 'active',  
    last_heartbeat DATETIME,  
    FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id) ON DELETE CASCADE  
);
```

```
-- -----  
-- 8. TABLE ANTICHEAT_LOG  
-- -----
```

```
CREATE TABLE IF NOT EXISTS anticheat_log (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    session_id INT NOT NULL,  
    utilisateur_id INT NOT NULL,  
    event_type VARCHAR(50) NOT NULL,  
    event_data TEXT,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (session_id) REFERENCES examen_session(id) ON DELETE  
CASCADE,  
    FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id) ON DELETE CASCADE  
);
```

```
-- -----  
-- INDEX POUR OPTIMISER LES PERFORMANCES
```

```
-- -----  
CREATE INDEX idx_tentative_user ON tentative(utilisateur_id);  
CREATE INDEX idx_tentative_date ON tentative(date);  
CREATE INDEX idx_reponse_tentative ON reponse_utilisateur(tentative_id);  
CREATE INDEX idx_anticheat_user ON anticheat_log(utilisateur_id);  
CREATE INDEX idx_anticheat_session ON anticheat_log(session_id);  
CREATE INDEX idx_anticheat_created ON anticheat_log(created_at);
```

```
-- -----  
-- INSERTION DES DONNÉES INITIALES  
-- -----
```

```
INSERT INTO utilisateur (nom, prenom, email, mot_de_passe, role) VALUES  
(  
  'Admin', 'Master', 'admin@leti-qcm.com',  
  '$2y$10$92IXUNpkjO0rQQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', 'admin');
```

```
INSERT INTO categorie (nom, description, icone) VALUES  
(  
  'Informatique', 'Questions sur le développement et les réseaux', 'fa-laptop-code'),  
  ('Mathématiques', 'Calcul, algèbre et géométrie', 'fa-calculator'),  
  ('Culture générale', 'Histoire, géographie et société', 'fa-globe'),  
  ('Sciences', 'Physique, chimie et biologie', 'fa-flask'),  
  ('Sport', 'Football, basket et actualité sportive', 'fa-futbol');
```

ANALYSE DES BESOINS

1. BESOINS FONCTIONNELS

Les besoins fonctionnels décrivent l'ensemble des fonctionnalités que doit proposer l'application afin de répondre aux attentes des utilisateurs. Dans le cadre de ce projet de développement d'une application web de QCM en ligne, plusieurs fonctionnalités essentielles ont été identifiées et implémentées.

- **Gestion des utilisateurs :**

L'application permet aux utilisateurs de créer un compte de manière sécurisée. Lors de l'inscription, l'utilisateur doit fournir une adresse email valide, un nom, un prénom ainsi qu'un mot de passe respectant des critères de sécurité. Les mots de passe sont chiffrés avant leur stockage en base de données. Les utilisateurs peuvent également se connecter à leur compte via un formulaire sécurisé.

- **Gestion des QCM :**

Un utilisateur connecté peut passer un QCM en sélectionnant au moins trois catégories. Le système génère alors dix questions aléatoires. L'utilisateur dispose d'un timer par question. À la fin, le score est calculé et affiché avec le détail des réponses.

- **Consultation des résultats :**

L'utilisateur peut consulter son historique de QCM passés ainsi que ses statistiques personnelles (moyenne, meilleur score, progression).

- **Système anti-triche :**

L'application intègre plusieurs mécanismes anti-triche : plein écran obligatoire, détection des changements d'onglet, blocage du clic droit, blocage du copier/coller, blocage des raccourcis clavier (F12, Ctrl+U, etc.), timer côté serveur et score de suspicion.

- **Interface administrateur :**

L'administrateur peut gérer les utilisateurs (liste, blocage, suppression), gérer les questions (ajout, modification, suppression), consulter les statistiques globales et visualiser les logs anti-triche.

2. BESOINS NON FONCTIONNELS

Dans le cadre de ce projet, plusieurs exigences non fonctionnelles ont été prises en compte afin de garantir la qualité, la sécurité et la performance de l'application web.

- **Sécurité :**

Les mots de passe des utilisateurs sont chiffrés avant leur enregistrement en base de données. Les formulaires utilisent la méthode POST et des vérifications sont effectuées côté serveur. Des protections contre les injections SQL sont mises en place via les requêtes préparées.

- **Performance :**

L'application a été conçue pour offrir des temps de réponse rapides. Les requêtes SQL ont été optimisées et des index ont été créés sur les champs fréquemment utilisés.

- **Ergonomie et expérience utilisateur :**

Une attention particulière a été portée à l'interface utilisateur. Le site a été conçu pour être clair et intuitif, avec une navigation simple entre les différentes pages (accueil, QCM, historique, administration).

- **Compatibilité et responsive design :**

Le site a été développé en HTML, CSS et JavaScript pour être compatible avec les principaux navigateurs web. Un design responsive permet une utilisation sur ordinateur, tablette et smartphone.

- **Fiabilité :**

Des tests ont été réalisés tout au long du développement afin de vérifier le bon fonctionnement des fonctionnalités. Des messages d'erreur explicites guident l'utilisateur en cas de problème.

- **Maintenabilité :**

Le code a été organisé de manière structurée (séparation PHP, HTML, CSS, SQL) afin de faciliter sa compréhension et sa modification. Des commentaires ont été ajoutés pour permettre une meilleure lisibilité.

MÉTHODOLOGIE DE TRAVAIL

Dans le cadre de la réalisation de ce projet, nous avons adopté une approche inspirée de la méthode agile Scrum Cette méthodologie nous a permis d'organiser efficacement le travail en équipe, de suivre l'avancement du projet et de nous adapter aux différentes difficultés rencontrées.

Présentation de la méthode Scrum

Scrum est une méthode de gestion de projet agile qui repose sur un travail collaboratif et itératif. Le projet est découpé en plusieurs phases appelées sprints, durant lesquelles des fonctionnalités précises sont développées, testées et améliorées. Cette approche permet de livrer progressivement une application fonctionnelle tout en facilitant les ajustements en cours de développement.

Organisation du travail :

Le projet a été réalisé en groupe de 2 . Les tâches ont été réparties en fonction des compétences de chacun (backend, frontend, base de données). Une liste des fonctionnalités à développer (backlog) a été définie.

Mise en place des sprints :

Le travail a été organisé en plusieurs sprints : mise en place de la base de données, développement du système d'inscription/connexion, développement du QCM, mise en place de l'anti-triche, création de l'interface administrateur.

Suivi et communication :

Une communication régulière entre les membres du groupe a permis de suivre l'avancement des tâches et de résoudre rapidement les problèmes.

4. OUTILS DE DÉVELOPPEMENT

Langages et technologies :

- **PHP** : traitement des données côté serveur
- **HTML/CSS** : structure et mise en forme
- **JavaScript** : interactivité et détection anti-triche
- **SQL / MySQL** : gestion de la base de données

Environnement de développement :

- **Visual Studio Code** : éditeur de code
- **MAMP / XAMPP** : serveur local

Outils de test :

- **Postman** : tests des requêtes API
- **Navigateur (Chrome DevTools)** : tests et débogage

5. PERSPECTIVES D'ÉVOLUTION

1. Améliorations fonctionnelles :

- Ajout d'un système de badges et de niveaux (gamification)
- Génération automatique de questions par IA
- Export des résultats en PDF
- Mode examen avec webcam

2. Améliorations techniques :

- Renforcement de la sécurité
- Utilisation d'un framework moderne (Laravel, Symfony)
- Création d'une application mobile

3. Amélioration de l'interface :

- Design plus moderne et attractif
- Meilleure expérience utilisateur
- Optimisation mobile

6. CONCLUSION GÉNÉRALE

Ce projet de développement web nous a permis de concevoir et réaliser une application complète de QCM en ligne avec un système anti-triche intégré. L'application développée répond aux principaux besoins identifiés, notamment la gestion des comptes utilisateurs, le passage de QCM par catégories, la consultation des résultats et l'administration complète.

Sur le plan technique, ce projet nous a permis de mettre en pratique plusieurs compétences essentielles en développement web : PHP, HTML, CSS, JavaScript et SQL. La conception de la base de données (MCD, MLD et script SQL) a constitué une étape importante dans la structuration du projet.

L'utilisation de la méthode Scrum a facilité l'organisation du travail, la répartition des tâches et le suivi de l'avancement. Les outils de développement tels que Visual Studio Code ont contribué à la réalisation et aux tests de l'application.

Enfin, ce projet constitue une expérience enrichissante, tant sur le plan technique qu'organisationnel, et ouvre la voie à de nombreuses améliorations possibles.